Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 1:

Track A: The OS as a Resource Manager

Elaborated: gr. FAF-243
Poiata Calin

Verified:
Patricia Reitman

Chișinău – 2026

Table of Contents

System Identification 1
Part 1. Files and directories (File management) 1
1. Navigation and Inspection 1
2. Create, Copy, Rename, and Remove 2
3. Ownership and Permissions 2
Observe (Part 1 Answers) 2
Interesting Output Highlight (Part 1) 3
Part 2. Processes (Process management) 3
4. Process Listing 3
5. Managing Background Jobs 3
6. Process Introspection via /proc 3
Observe (Part 2 Answers) 4
Interesting Output Highlight (Part 2) 4
Part 3. Memory (Memory management) 4
Memory Inspection 4
Observe (Part 3 Answers) 5
Interesting Output Highlight (Part 3) 5
Part 4. Devices and storage (I/O management) 5
Storage and Device Listing 5
Observe (Part 4 Answers) 6
Interesting Output Highlight (Part 4) 6
Wrap-up and Deliverable: Closing Synthesis 6
Conclusions 8

System Identification

```
❱ whoami
calin
❱ uname -a
Linux archlinux 7.2.7-arch1-1 #1 SMP PREEMPT_DYNAMIC Mon, 21 Sep 2026 18:51:14 +0000
x86_64 GNU/Linux
❱ uptime
 16:35:12 up 2:15,  1 user,  load average: 1.82, 1.45, 1.10
```

Part 1. Files and directories (File management)

1. Navigation and Inspection

```
❱ pwd
/home/calin/code/os-labs/lab1
❱ ls -la /
total 68
dr-xr-xr-x  17 root root  4096 Oct 20  2025 .
dr-xr-xr-x  17 root root  4096 Oct 20  2025 ..
lrwxrwxrwx   1 root root     7 Oct 12  2025 bin -> usr/bin
drwxr-xr-x   4 root root  4096 Jan  1  1970 boot
drwxr-xr-x  20 root root  4200 Oct  5 15:16 dev
drwxr-xr-x 130 root root 12288 Oct  5 15:34 etc
drwxr-xr-x   4 root root  4096 Sep 23  2024 home
lrwxrwxrwx   1 root root     7 Oct 12  2025 lib -> usr/lib
lrwxrwxrwx   1 root root     7 Oct 12  2025 lib64 -> usr/lib
drwx------   2 root root 16384 Sep 23  2024 lost+found
drwxr-xr-x   2 root root  4096 Apr  7  2024 mnt
drwxr-xr-x  14 root root  4096 Oct  1 15:17 opt
dr-xr-xr-x 372 root root     0 Oct  5 15:16 proc
drwx------   8 root root  4096 Oct  1 15:20 root
drwxr-xr-x  38 root root   940 Oct  5 15:16 run
lrwxrwxrwx   1 root root     7 Oct 12  2025 sbin -> usr/bin
drwxr-xr-x   5 root root  4096 Jul  6 16:29 srv
dr-xr-xr-x  13 root root     0 Oct  5 15:16 sys
drwxrwxrwt  25 root root   820 Oct  5 15:53 tmp
drwxr-xr-x  10 root root  4096 Oct  5 15:34 usr
drwxr-xr-x  12 root root  4096 Oct  3 21:01 var
❱ ls -la ~
total 312
drwx------ 42 calin calin  4096 Oct  5 15:42 .
drwxr-xr-x  4 root  root   4096 Sep 23  2024 ..
drwxr-xr-x  3 calin calin  4096 Aug 25 13:04 .antigravity-ide
drwxr-xr-x  5 calin calin  4096 Aug 25 13:04 .ansible
-rw-------  1 calin calin   575 Nov 28  2025 .bash_history
-rw-r--r--  1 calin calin    21 Sep 23  2024 .bash_logout
-rw-r--r--  1 calin calin    57 Sep 23  2024 .bash_profile
-rw-r--r--  1 calin calin   202 Sep 23  2024 .bashrc
drwxr-xr-x 26 calin calin  4096 Oct  1 15:19 .cache
drwxr-xr-x 12 calin calin  4096 Sep 28 16:09 .config
drwxr-xr-x  2 calin calin  4096 Sep 28 16:45 Desktop
drwxr-xr-x  4 calin calin  4096 Sep 28 17:13 Documents
drwxr-xr-x  2 calin calin  4096 Sep 28 16:30 Downloads
drwxr-xr-x  7 calin calin  4096 Oct  3 14:09 .gemini
-rw-r--r--  1 calin calin   399 Oct  2 16:19 .gitconfig
drwx------  5 calin calin  4096 Oct 24  2024 .gnupg
-rw-r--r--  1 calin calin   345 Sep 19 13:55 .gtkrc-2.0
drwxr-xr-x  7 calin calin  4096 Feb  3  2026 .local
drwx------  5 calin calin  4096 Oct  1 12:46 .mozilla
drwxr-xr-x  4 calin calin  4096 Jul 10  2025 Music
drwxr-xr-x  5 calin calin  4096 Sep 25  2025 .npm
drwxr-xr-x  8 calin calin  4096 Aug 14 11:56 Pictures
drwxr-x---  3 calin calin  4096 Oct  3 17:07 Projects
drwxr-xr-x  2 calin calin  4096 Sep 23  2024 Public
drwx------  2 calin calin  4096 Aug 19 11:37 .ssh
drwxr-xr-x  2 calin calin  4096 Apr 29 11:47 .steam
drwxr-xr-x  2 calin calin  4096 Sep 23  2024 Templates
drwxr-xr-x  5 calin calin  4096 Aug 26 15:45 Videos
-rw-------  1 calin calin 12297 Sep 28 16:48 .viminfo
```

```
drwxr-xr-x  4 calin calin  4096 Nov 14  2024 .vscode
drwxr-xr-x  4 calin calin  4096 Apr  8 15:52 .wine
-rw-------  1 calin calin 133070 Oct 5 15:42 .zsh_history
-rw-r--r--  1 calin calin  4142 Aug 25 12:50 .zshrc
❱ cd /etc && ls | head
alsa
anacrontab
arch-release
arp-scan
arptables.conf
audisp
audit
avahi
bash.bash_logout
bash.bashrc
```

2. Create, Copy, Rename, and Remove

```
❱ cd /home/calin/code/os-labs/lab1
❱ mkdir -p demo && cd demo
❱ echo "hello operating systems" > note.txt
❱ cp note.txt copy.txt
❱ mv copy.txt renamed.txt
❱ ls -l
total 8
-rw-r--r-- 1 calin calin 24 Sep 28 16:36 note.txt
-rw-r--r-- 1 calin calin 24 Sep 28 16:36 renamed.txt
❱ rm renamed.txt
❱ ls -l
total 4
-rw-r--r-- 1 calin calin 24 Sep 28 16:36 note.txt
```

3. Ownership and Permissions

```
❱ ls -l note.txt
-rw-r--r-- 1 calin calin 24 Sep 28 16:36 note.txt
❱ chmod 600 note.txt
❱ ls -l note.txt
-rw------- 1 calin calin 24 Sep 28 16:36 note.txt
❱ chmod 644 note.txt
❱ ls -l note.txt
-rw-r--r-- 1 calin calin 24 Sep 28 16:36 note.txt
```

Observe (Part 1 Answers)

1. **Who owns the files you just created, and which group?**
   - **Owner:** `calin` (user ID 1000).
   - **Group:** `calin` (group ID 1000).
   - The file created is owned by the user and the group with the same name, in my case `calin`. The operating system automatically assigns the credentials of the active user session as the owner and primary group of newly created files.

2. **What do the ten characters at the start of ls -l mean for note.txt after chmod 600?**
   - The string is `-rw-------`.
     - **1st character (`-`):** File type indicator, showing this is a standard regular file (not a directory `d` or symbolic link `l`).
     - **Characters 2–4 (`rw-`):** User/Owner permissions. Read (`r`) and write (`w`) permissions are granted to the owner (`calin`), while execute (`-`) is denied.
     - **Characters 5–7 (`---`):** Group permissions. Read, write, and execute permissions are all denied.
     - **Characters 8–10 (`---`):** Others/World permissions. Read, write, and execute permissions are all denied.
   - In octal, `600` represents binary `110 000 000`, indicating that only the owner of the file can read or write to the file, with the superuser being an exception.

3. **Name two directories directly under / and say, in one line each, what they hold.**
   - `/etc` : Contains system-wide configuration files and initialization scripts (for example, `fstab` holding the mounted filesystems).
   - `/usr` : Holds installed software, user binaries, libraries, and other read-only data that isn't essential when booting the system.

Interesting Output Highlight (Part 1)

Executing `echo "hello operating systems" > note.txt` produces no stdout output directly in the terminal, but showcases the shell stream redirection mechanism—directly creating and streaming text into a persistent filesystem node without invoking an interactive text editor.

Part 2. Processes (Process management)

4. Process Listing

```
❱ ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0  22356 14208 ?        Ss   15:16   0:01 /usr/lib/systemd/
systemd --switched-root --system --deserialize=36
root           2  0.0  0.0      0     0 ?        S    15:16   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    15:16   0:00
[pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   15:16   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   15:16   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   15:16   0:00 [kworker/R-
slub_flushwq]
root           7  0.0  0.0      0     0 ?        I<   15:16   0:00 [kworker/R-netns]
root           8  0.0  0.0      0     0 ?        I<   15:16   0:00 [kworker/0:0H-kblockd]
root           9  0.0  0.0      0     0 ?        I    15:16   0:00 [kworker/u32:0-
events_unbound]
❱ ps aux | wc -l
361
```

5. Managing Background Jobs

```
❱ sleep 300 &
[1] 34812
❱ jobs
[1]+  Running                 sleep 300 &
❱ ps -ef | grep sleep
calin      34812   34105  0 16:36 pts/1    00:00:00 sleep 300
calin      34814   34105  0 16:36 pts/1    00:00:00 grep sleep
❱ kill 34812
[1]+  Terminated              sleep 300
```

6. Process Introspection via /proc

```
❱ sleep 300 &
[1] 34820
❱ PID=34820
❱ ls /proc/$PID/
arch_status         fdinfo              ns                  smaps_rollup
attr                gid_map             numa_maps           stack
autogroup           io                  oom_adj             stat
auxv                ksm_merging_pages   oom_score           statm
cgroup              ksm_stat            oom_score_adj       status
clear_refs          limits              pagemap             syscall
cmdline             loginuid            personality         task
comm                map_files           projid_map          timens_offsets
coredump_filter     maps                root                timers
cpu_resctrl_groups  mem                 sched               timerslack_ns
cwd                 mountinfo           schedstat           uid_map
environ             mounts              sessionid           wchan
exe                 mountstats          setgroups
```

```
fd                  net                 smaps
❱ cat /proc/$PID/status | head -20
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   34820
Ngid:   0
Pid:    34820
PPid:   34105
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 998 1000
NStgid: 34820
NSpid:  34820
NSpgid: 34105
NSsid:  34105
Kthread:        0
VmPeak:     6140 kB
VmSize:     6140 kB
VmLck:         0 kB
❱ kill $PID
```

Observe (Part 2 Answers)

1. **What is the PID of process number 1, and what is it (look it up)?**
   - **PID:** 1.
   - **Program:** `systemd` (`/usr/lib/systemd/systemd`).
   - The process with PID 1 is `systemd`, which is the system and service manager of my Linux machine, responsible for initializing the system, starting all the other services and processes, and supervising them throughout execution.
2. **Roughly how many processes were running on your idle VM?**
   - My machine has **361 running processes** (determined via `ps aux | wc -l`). A substantial portion of these represent internal kernel threads (such as `[kthreadd]` and `[kworker/...]`) that assist the OS in background work and hardware handling, alongside systemd background services and user session tasks.
3. **From /proc//status, what does the "State:" line say for your sleeping process?**
   - **State: S (sleeping).** The process is sleeping, not actively executing instructions on the CPU. In this state it waits for an event—in this case, the expiry of the 300 seconds timer—consuming zero processor cycles.

Interesting Output Highlight (Part 2)

The command pipeline `ps aux | wc -l` (returning 361) provides a clever Unix way of obtaining the total active process count by streaming process data directly into a line counter.

Part 3. Memory (Memory management)

Memory Inspection

```
❱ free -h
               total        used        free      shared  buff/cache   available
Mem:            14Gi       6.5Gi       4.7Gi       162Mi       4.2Gi       8.3Gi
Swap:          4.0Gi          0B       4.0Gi
❱ cat /proc/meminfo | head -6
MemTotal:       15452208 kB
MemFree:         4928512 kB
MemAvailable:    8703180 kB
Buffers:          182400 kB
```

```
Cached:          4225700 kB
SwapCached:            0 kB
❱ sleep 300 & PID=$!
[1] 34850
❱ grep VmRSS /proc/$PID/status
VmRSS:      4096 kB
❱ kill $PID
```

Observe (Part 3 Answers)

1. **How much total RAM does the VM have, and how much is free?**
   - **Total RAM:** I have a total of 14 GiB (approx. 15.4 GB total physical memory).
   - **Free RAM:** 4.7 GiB is strictly free/unallocated (`MemFree`), while 8.3 GiB is reported as available (`MemAvailable`). The available metric reflects true usable memory because the OS can instantly evict reclaimable page caches and file buffers (`buff/cache` : 4.2 GiB) when applications demand memory.
2. **What is swap, and how much is configured?**
   - **What swap is:** Swap is an "extension" of the RAM that will take some of the load when RAM is full, allowing the kernel to page out inactive memory segments to disk or compressed memory.
   - **Configured swap:** Currently 4.0 GiB configured (with 0B used and 4.0 GiB free).
3. **How much resident memory (VmRSS) does a bare sleep process use, and does that surprise you?**
   - **Resident memory:** Roughly 4 MB (`4096 kB`).
   - **Reflection:** It is quite surprising given that it doesn't do anything but wait for an event to trigger it. The memory usage is driven by dynamically linked C runtime libraries (`libc.so`), the runtime linker (`ld-linux.so`), initial stack mappings, and kernel page table structures allocated for process isolation.

Interesting Output Highlight (Part 3)

Using `free -h` formats the raw byte figures into human-readable GiB and MiB notations (`-h` literally stands for "human-readable"), making system memory state significantly easier to interpret at a glance.

Part 4. Devices and storage (I/O management)

Storage and Device Listing

```
❱ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p2   63G   43G   18G  71% /
devtmpfs        7.4G     0  7.4G   0% /dev
tmpfs           7.5G   40M  7.4G   1% /dev/shm
efivarfs        154K   72K   78K  49% /sys/firmware/efi/efivars
tmpfs           3.0G  2.1M  3.0G   1% /run
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-networkd.service
tmpfs           7.5G   43M  7.4G   1% /tmp
/dev/nvme0n1p1 1022M  279M  744M  28% /boot
/dev/nvme0n1p3  405G  184G  200G  48% /home
tmpfs           1.5G  196K  1.5G   1% /run/user/1000
❱ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
zram0       253:0    0     4G  0 disk [SWAP]
nvme0n1     259:0    0 476.9G  0 disk
├─nvme0n1p1 259:1    0     1G  0 part /boot
├─nvme0n1p2 259:2    0    64G  0 part /
```

```
└─nvme0n1p3 259:3    0 411.9G  0 part /home
❱ du -sh /home/calin/code/os-labs/lab1
16K     /home/calin/code/os-labs/lab1
❱ ls -l /dev | head
total 0
crw-r--r--   1 root  root    10,   235 Oct  5 15:31 autofs
drwxr-xr-x   2 root  root          140 Oct  5 15:31 block
crw-rw----   1 root  disk    10,   234 Oct  5 15:31 btrfs-control
drwxr-xr-x   3 root  root           60 Oct  5 15:16 bus
drwxr-xr-x   2 root  root         4600 Oct  5 15:31 char
crw-------   1 root  root     5,     1 Oct  5 15:31 console
lrwxrwxrwx   1 root  root           11 Oct  5 15:16 core -> /proc/kcore
drwxr-xr-x  10 root  root          200 Oct  5 15:16 cpu
crw-------   1 root  root    10,   259 Oct  5 15:31 cpu_dma_latency
❱ mount | head
/dev/nvme0n1p2 on / type ext4 (rw,relatime)
devtmpfs on /dev type devtmpfs
(rw,nosuid,size=7726104k,nr_inodes=1931526,mode=755,inode64,huge=advise)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,huge=advise,usrquota)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
efivarfs on /sys/firmware/efi/efivars type efivarfs (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
```

Observe (Part 4 Answers)

1. **Which device is your root filesystem "/" mounted on?**
   - The root filesystem is mounted on `/dev/nvme0n1p2`.
2. **Give one entry from /dev and say what real thing it stands for.**
   - `nvme0n1` is the physical solid-state drive (SSD), partitioned into distinct volumes such as root (`/dev/nvme0n1p2`) and the home partition (`/dev/nvme0n1p3`). Another entry, `/dev/console`, represents the primary system console character device for interactive terminal I/O.
3. **In one sentence: what does "everything is a file" mean, based on what you saw?**
   - "Everything is a file" means that each piece of a system, including how it interacts with hardware devices, memory, and runtime kernel state, is configured and accessed via a file-like interface within a single unified filesystem namespace.

Interesting Output Highlight (Part 4)

Inspecting `df -h` revealed the large variety of virtual filesystems mounted simultaneously alongside the SSD storage (such as `devtmpfs`, `efivarfs`, and multiple `tmpfs` mounts), highlighting how the operating system seamlessly integrates memory-backed kernel structures and physical block devices.

Wrap-up and Deliverable: Closing Synthesis

The operating system acts as the foundational resource manager of a computer by coordinating four core hardware resources:

1. **CPU (Processes):** The OS allocates and schedules CPU time among concurrently executing tasks. Process listing and lifecycle controls (managed and monitored with utilities like `ps`, `htop`, and `/proc/<pid>/status`) demonstrate how the kernel isolates, tracks, and manages execution state.
2. **RAM (Physical Memory):** Physical memory is balanced dynamically across processes, disk caches, and virtual memory. As observed via `free -h` and `/proc/meminfo`, the kernel prioritizes active working sets while utilizing inactive memory for high-speed page caching and swapping.

3. **Storage (Files and Directories):** The OS translates raw block devices into structured, hierarchical trees. Storage usage and partition health are tracked via commands like `df -h`, while permission structures (`chmod`, `ls -la`) enforce rigorous multi-user access boundaries.
4. **I/O Devices:** Hardware devices (from USB controllers to block storage) are exposed as uniform character or block device nodes under `/dev` and enumerated through tools like `lsblk` and `lsusb`, enabling programs to communicate with physical peripherals through standard file I/O operations.

Through these four pillars, the operating system effectively abstracts physical hardware complexity, providing a predictable, secure, and robust execution environment.

Conclusions

This laboratory work provided a comprehensive hands-on exploration of the fundamental mechanisms through which the operating system fulfills its central mission as a hardware resource manager. By examining the system through native Unix inspection utilities, the core responsibilities of the Linux kernel—specifically arbitrating processor execution, structuring physical memory, maintaining storage hierarchies, and abstracting device communication—were systematically verified. Rather than operating as an opaque layer, the kernel was shown to implement robust abstractions that isolate individual applications while coordinating access to shared underlying physical components.

Throughout the practical exercises, key design philosophies of the Unix architecture were observed in direct operation. The universal principle that "everything is a file" was clearly demonstrated through virtual filesystems such as `/proc` and device nodes under `/dev`, illustrating how heterogeneous hardware and live kernel control structures are represented seamlessly within a unified hierarchical namespace. Furthermore, memory management inspections highlighted the sophisticated balance maintained between active user allocations, reclaimable page caches, and swap space, underscoring how modern operating systems optimize hardware throughput without sacrificing system responsiveness or process stability.

Ultimately, performing direct process introspection and filesystem permission auditing provided critical practical insights that bridge abstract operating system theory with real-world system behavior. Analyzing attributes such as Process Control Block entries, file descriptor tables, and resident set sizes (`VmRSS`) demystified the underlying overhead involved in basic task execution. This investigation solidifies an essential foundational understanding of low-level system engineering, establishing the necessary analytical groundwork for future study into system calls, concurrent process synchronization, and custom kernel development.